实验报告：逻辑回归与大学排名分类

项目	内容
课程名称	人工智能数学原理与算法 B
实验名称	第三次实验：逻辑回归与大学排名分类
姓名	吴炳睿
学号	PB24081512
实验日期	2026 年 10 月 19日
主讲 / 整理 / 任课教师	胡泷 / 胡泷、曾有成 / 钟志诚
单位	中国科学技术大学 人工智能与数据科学学院

一、实验目的

1. 理解逻辑回归从增广输入与参数的内积得到线性得分，再经 sigmoid 函数转换为概率，并按阈值给出预测类别的完整过程，区分得分、概率和类别三种输出。
2. 掌握混合类型表格数据的预处理方法，包括缺失标签过滤、数值字符串清洗、训练集均值填补、标准化、类别独热编码，以及缺失和未见类别的 Unknown 处理。
3. 理解二元交叉熵与 L1/L2 正则化的含义，通过手算和程序对照一次梯度更新，掌握小批量随机梯度下降的训练步骤。
4. 明确训练集、验证集和测试集的职责，只依据验证集选择模型，并在冻结模型后使用测试集进行一次性评价；结合混淆矩阵、多数类基线和多项指标解释分类结果。

二、实验环境与数据

2.1 实验环境

表 1 实验环境

项目	内容
操作系统	Windows；Python 环境报告为 Windows 10，宿主机为 Windows 11
Python 版本	3.11.16
PyTorch / pandas / NumPy 版本	2.14.0+cpu / 3.0.5 / 2.4.6
运行设备	CPU，Intel64 Family 6 Model 186 Stepping 2
运行入口	`logic.ipynb`，结果目录为 `code/outputs-qs-notebook/`

2.2 实验数据

实验数据为 QS 2026 世界大学排名的课堂整理版，于 2026-10-08 从公开第三方镜像获取。原始表共 1504 行、30 列，学校名无重复。标签 `top500` 定义为源排名小于或等于 500 时为 1，其余为 0。由于存在并列名次，全表正类共 502 所。数据按类别分层划分为训练集、验证集和测试集，划分种子为 20261008，三份数据互斥且保持自然类别比例。

本任务只使用部分同年已公布指标识别同年榜单前 500 名，不预测未来年份排名，也不评价源表之外的学校。部分输入分数本身参与或关联 QS 排名构造，因此结果应理解为教学分类结果，不能作因果解释。

表 2 数据划分与缺失情况

划分	样本数	正类数	正类比例	真实缺失情况
训练集	902	301	33.37%	status 缺失 31；国际教师分数缺失 49；国际学生分数缺失 18
验证集	300	100	33.33%	status 缺失 13；国际教师分数缺失 20；国际学生分数缺失 12
测试集	302	101	33.44%	status 缺失 4；size 缺失 1；research 缺失 1；国际教师分数缺失 18；国际学生分数缺失 7

三个标签文件中的标签均有效。标签缺失处理仅通过 C1 的小例子练习，不在主实验中人为制造缺失或删除标签。

模型输入采用字段白名单。`name` 仅用于追溯，若作为输入容易让模型记忆学校名称，且无法泛化到其他学校；`row_id` 由原始表行号保留而来，与排名排序相关，直接输入会造成信息泄漏。`top500` 是标签，所有当年排名、总分、单项名次、上一年排名和完整评分向量均被排除。

输入包含 3 个数值字段和 5 个类别字段。数值字段使用训练集非缺失均值填补，并除以训练集总体标准差（`ddof=0`）；常数列尺度记为 1，全空训练列均值记为 0。类别字段的类别顺序由训练集按字母排序确定，最后追加 `Unknown`；缺失值和验证、测试中未见的类别均映射到 `Unknown`。完整组共 3 个数值特征和 25 个独热特征，合计 28 维；在首列增加常数 1 作为截距后，模型输入为 29 维。

三、实验原理

设第 n 个样本的增广特征为 $\tilde{\mathbf{x}}^{(n)} \in \mathbb{R}^{d+1}$，参数为 $\boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_d)^\top$。线性得分为

$$z^{(n)} = (\tilde{\mathbf{x}}^{(n)})^\top \boldsymbol{\theta}, \qquad p^{(n)} = \sigma(z^{(n)}) = \frac{1}{1 + e^{-z^{(n)}}}.$$

其中 θ_0 是截距，模型保存为形状 $(d+1, 1)$ 的列向量。概率大于或等于阈值 τ 时预测为 1，否则预测为 0。本实验固定主阈值 $\tau = 0.5$。

二元交叉熵由伯努利似然的负对数得到。对一个批次，平均数据损失为

$$R_B(\boldsymbol{\theta}) = -\frac{1}{B} \sum_{n=1}^{B} \left[y^{(n)} \log p^{(n)} + (1 - y^{(n)}) \log(1 - p^{(n)}) \right].$$

程序使用 `nn.BCEWithLogitsLoss` 直接接收原始得分，在内部稳定地完成 sigmoid 与交叉熵计算，避免先取概率再计算对数造成数值问题。

L1 和 L2 惩罚分别定义为

$$\Omega_{\mathrm{L1}}(\boldsymbol{\theta}) = \sum_{j=1}^{d} |\theta_j|, \qquad \Omega_{\mathrm{L2}}(\boldsymbol{\theta}) = \frac{1}{2} \sum_{j=1}^{d} \theta_j^2.$$

求和从 $j = 1$ 开始，因此截距 θ_0 不受惩罚。训练总目标为

$$J_B(\boldsymbol{\theta}) = R_B(\boldsymbol{\theta}) + \lambda \Omega(\boldsymbol{\theta}),$$

其中 λ 对应代码中的 `strength`。小批量 SGD 的更新式为

$$\boldsymbol{\theta}^{[t+1]} = \boldsymbol{\theta}^{[t]} - \eta \nabla_{\boldsymbol{\theta}} J_B(\boldsymbol{\theta}^{[t]}),$$

η 为学习率。评价阶段由 TN、FP、FN、TP 四种计数计算准确率、精确率、召回率和 F1：

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP},$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2\,\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

四、实验内容与步骤

4.1 完成情况

运行 `python checkpoints.py all` 后，C0–C7 均输出 `PASS`。各检查点的小例子数值和完整实验结果如下文所示。

表 3 各检查点完成情况

检查点	内容	完成状态	存在的问题
C0	数据形状与矩阵乘法	已完成	无
C1	标签过滤与数值清洗	已完成	无
C2	数值标准化与独热编码	已完成	无
C3	数据集与批次划分	已完成	无
C4	线性得分与概率	已完成	无
C5	损失函数与正则化	已完成	无
C6	训练循环与参数更新	已完成	无
C7	评价指标与模型选择	已完成	无

4.2 各检查点的实现与验证

C0：数据形状与矩阵乘法

- 实现位置与思路：该阶段观察已提供的矩阵乘法样例和训练表形状，不补写函数。样本按行排列，因此矩阵乘法的每一行对应一个样本的得分。

- 关键代码：

```
X = torch.tensor([[1., 2.], [1., -1.]])
w = torch.tensor([[0.], [1.]])
score = X @ w
```

- 验证样例：原权重 $(0,1)^\top$ 时，预期得分为 $(2,-1)^\top$；将权重改为 $(1,2)^\top$ 后，手算第一行 $1+4=5$、第二行 $1-2=-1$。
- 实际输出与分析：原权重的输出为 `[[2.0], [-1.0]]`，修改权重后为 `[[5.0], [-1.0]]`；课堂训练表形状为 `(902, 11)`，缺失标签数为 0。原始表有 11 列，但其中包含名称、行号和标签等不能直接输入模型的字段，类别字段还需要展开为独热列，因此模型输入不是 11 维。

C1：标签过滤与数值清洗

- 实现位置与思路：`filter_labels` 只删除标签缺失的行，并重置行索引；`clean_numeric` 使用 `pd.to_numeric(..., errors="coerce")` 把无法解析的文本转为缺失值。特征缺失不在这一步填补。
- 关键代码：

```
return frame.dropna(subset=["top500"]).reset_index(drop=True)

return pd.to_numeric(series, errors="coerce")
```

- 验证样例：标签为 `[1, NaN, 0]`、特征为 `[None, 2, 3]` 的三行数据；以及数值列 `["12.5", "-", "", None]`。
- 实际输出与分析：过滤后保留标签 `[1, 0]`，特征为 `[NaN, 2]`，原输入表仍保持 3 行；数值清洗结果为 `[12.5, NaN, NaN, NaN]`。字符串 `"0"` 和数值 `0` 都保留为 `0.0`，因为 0 是合法分数，不能与缺失混淆。

C2：数值标准化与独热编码

- 实现位置与思路：`fit_statistics` 在训练集上计算均值和总体标准差，并用均值填补缺失后计算标准差；零标准差替换为 1。`apply_statistics` 只应用已保存的统计量，不在验证集上重新拟合。`fit_categories` 按字段收集训练非缺失类别、排序并追加 `Unknown`；`encode_categories` 把缺失和未见值映射到 `Unknown`。`to_tensors` 转为 `float32` 并添加截距列。
- 关键代码：

```
means = features.mean().fillna(0.0)
scales = features.fillna(means).std(ddof=0).replace(0.0, 1.0).fillna(1.0)

values = frame[column].fillna("Unknown")
values = values.where(values.isin(levels), "Unknown")

X = torch.cat([torch.ones((X.shape[0], 1)), X], dim=1)
```

- 验证样例：训练特征 `x=[1,3,NaN]`、常数列 `[5,5,5]`、全空列 `[NaN,NaN,NaN]`；验证值 `x=100` 和缺失值。类别字段为 `Asia`、`Europe`、缺失；验证行依次为 `Europe`、`Atlantis`、缺失。
- 实际输出与分析：训练均值为 `[2.0, 5.0, 0.0]`，尺度为 `[0.8164966, 1.0, 1.0]`；验证值 100 标准化为 `120.0250`，缺失值填补后标准化为 `0.0`。类别表为 `["Asia", "Europe", "Unknown"]`，三行独热结果依次为 `(0,1,0)`、`(0,0,1)`、`(0,0,1)`。类别顺序由训练状态固定，转换时索引保持不变，训练类别表也不会被修改。完整特征为 28 维，添加首列常数后为 29 维。

C3：数据集与批次划分

- 实现位置与思路：`UniversityDataset.__len__` 返回 `len(self.X)`；`__getitem__` 用同一个索引返回 `(X[index], y[index])`；`make_loader` 只在训练时打乱，并设置 `drop_last=False`。
- 关键代码：

```
return len(self.X)

return (self.X[index], self.y[index])

return DataLoader(
    dataset,
    batch_size=batch_size,
    shuffle=training,
    drop_last=False,
    num_workers=0,
)
```

- 验证样例：`X = torch.arange(15).reshape(5, 3)`，`y = torch.arange(5).reshape(5, 1)`，批量大小为 2。
- 实际输出与分析：样本数为 5；索引 2 返回特征 `[6, 7, 8]` 和标签 `[2]`，两者保持配对；批次大小依次为 `[2, 2, 1]`。真实训练集有 902 条样本，`902 = 14×64 + 6`，因此共有 15 批，最后一批保留 6 条样本。

C4：线性得分与概率

- 实现位置与思路：`sigmoid` 对每个线性得分逐元素应用 sigmoid 公式；`LogisticRegression.__init__` 把零初始化的 `(input_dim,1)` 张量注册为 `nn.Parameter`；`forward` 返回矩阵乘法得到的原始得分，不在这里调用 sigmoid。
- 关键代码：

```
return 1 / (1 + torch.exp(-z))

self.theta = nn.Parameter(torch.zeros((input_dim, 1)))

return X @ self.theta
```

- 验证样例：得分为 `[-2, 0, 2]`；增广输入为 `(1,3,2)`，参数为 `(1,2,-1)^T`。
- 实际输出与分析：三个得分对应的概率约为 `[0.119203, 0.500000, 0.880797]`。输入与参数分别为 `(1,3)` 和 `(3,1)`，矩阵乘法结果为 `(1,1)`，得分为 `5.0`，概率为 `0.993307`。`@` 是矩阵乘法，按行计算特征加权和；`*` 是逐元素乘法，形状和计算含义都不同。

C5：损失函数与正则化

- 实现位置与思路：`data_loss` 使用 `nn.BCEWithLogitsLoss()` 接收原始得分并返回平均损失；`penalty` 取 `theta[1:]`，分别计算 L1、L2 惩罚，`none` 返回与参数计算图相连的零值。
- 关键代码：

```
return nn.BCEWithLogitsLoss()(logits, y)

w = self.theta[1:]
if kind == "none":
    return self.theta.sum() * 0.0
if kind == "l1":
    return w.abs().sum()
if kind == "l2":
    return 0.5 * (w ** 2).sum()
```

- 验证样例：两个样本的得分均为 0，标签为 `[1, 0]`；参数为 `(10,3,-4)^T`；另取得分 `[1000,-1000]`、标签 `[0,1]` 检查极端值。
- 实际输出与分析：零得分时平均损失为 `0.693147`，等于 $\log 2$；极端值的平均损失为 `1000.0000`，仍为有限数。参数 `(10,3,-4)^T` 的 L1 惩罚为 $3+4=7$，L2 惩罚为 $0.5(3^2+(-4)^2)=12.5$。L2 对截距的梯度为 0，对两个特征权重的梯度为

`[3, -4]`。截距只平移整体得分，对特征权重施加惩罚不会带来同等的正则效果，因此不惩罚截距。

C6：训练循环与参数更新

- 实现位置与思路：每个批次依次清除旧梯度、前向计算得分、计算数据损失、加上正则项形成总目标、反向传播并更新参数；随后以实际样本数加权累计数据损失。
- 关键代码：

```
optimizer.zero_grad()
logits = model(X)
loss = data_loss(logits, y)
objective = loss + strength * model.penalty(kind)
objective.backward()
optimizer.step()
total += loss.detach().item() * len(y)
count += len(y)
```

- 手算过程：取

$$\mathbf{X} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \boldsymbol{\theta}^{[0]} = (0, 0, 0)^\top.$$

初始两个得分均为 0，概率均为 0.5，因此 $\mathbf{p} - \mathbf{y} = (-0.5, 0.5)^\top$。平均梯度为

$$\frac{1}{B}\mathbf{X}^\top(\mathbf{p} - \mathbf{y}) = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}\begin{pmatrix} -0.5 \\ 0.5 \end{pmatrix} = (0, -0.25, 0.25)^\top.$$

学习率为 0.4 时，第一次更新为

$$\boldsymbol{\theta}^{[1]} = (0, 0, 0)^\top - 0.4(0, -0.25, 0.25)^\top = (0, 0.1, -0.1)^\top.$$

- 程序输出与对照分析：第一次训练的数据损失为 `0.693147`，更新后参数为 `(0.000000, 0.100000, -0.100000)`，与手算一致。第二次训练时，参数对应的两个概率约为 `0.524979` 和 `0.475021`，更新后参数约为 `(0, 0.195004, -0.195004)`，第二次数据损失为 `0.644397`。若在批次开始时遗漏 `zero_grad()`，第二次更新会继续累加第一次的梯度，结果将与手算不符。

C7：评价指标与模型选择

- 实现位置与思路：`binary_metrics` 先把标签和概率转为一维；概率大于或等于阈值时预测为 1；再用布尔运算统计 TP、TN、FP、FN，并由四种计数计算四项指标。全部实验只用验证集比较正则化方案，冻结后才读取测试集。

- 关键代码：

```
truth = y.reshape(-1).bool()
prediction = probabilities.reshape(-1) >= threshold

tp = int((truth & prediction).sum().item())
tn = int((~truth & ~prediction).sum().item())
fp = int((~truth & prediction).sum().item())
fn = int((truth & ~prediction).sum().item())
```

- 验证样例：`y=[1,0,1,0]`，`p=[0.9,0.8,0.4,0.1]`，阈值分别为 0.5 和 1.0。
- 实际输出与分析：阈值 0.5 时预测为 `[1,1,0,0]`，TN、FP、FN、TP 均为 1；准确率、精确率、召回率和 F1 均为 `0.5`。阈值 1.0 时所有概率都小于阈值，TP 和 FP 为 0，因此精确率为 `0.0`，体现了“没有预测正类”时的指标约定。完整验证集的不同阈值结果见第五部分。

五、实验结果

5.1 实验设置

表 4 训练设置

设置项	取值
数据划分种子 / 训练种子	20261008 / 42
参数初始化方式	全零初始化
优化器与学习率	小批量 SGD，学习率 0.05
批量大小 / 最大轮数	64 / 60
正则化方案与系数	比较无正则（0）、L2（0.01）与 L1（0.01）
决策阈值	0.5
模型选择规则	每种方案取验证数据损失最低的轮次，再在三种方案中选择验证损失最低者

5.2 验证集比较与模型冻结

表 5 三种正则化方案的验证集结果

方案	正则系数	最佳轮次	验证数据损失	准确率	精确率	召回率	F1 分数
无正则	0	60	0.371792	0.8300	0.7753	0.6900	0.7302
L2	0.01	60	0.379552	0.8300	0.7816	0.6800	0.7273
L1	0.01	60	0.392744	0.8300	0.7816	0.6800	0.7273

在读取测试集之前，冻结无正则方案：`kind="none"`、`strength=0`、学习率 0.05、批量大小 64、训练种子 42、最大轮数 60、最佳轮次 60，固定阈值 0.5，使用完整特征集。选择依据是三种方案中该方案的验证数据损失最低（0.371792），而不是测试集表现。三者的验证准确率相同，但无正则方案的召回率和 F1 略高。

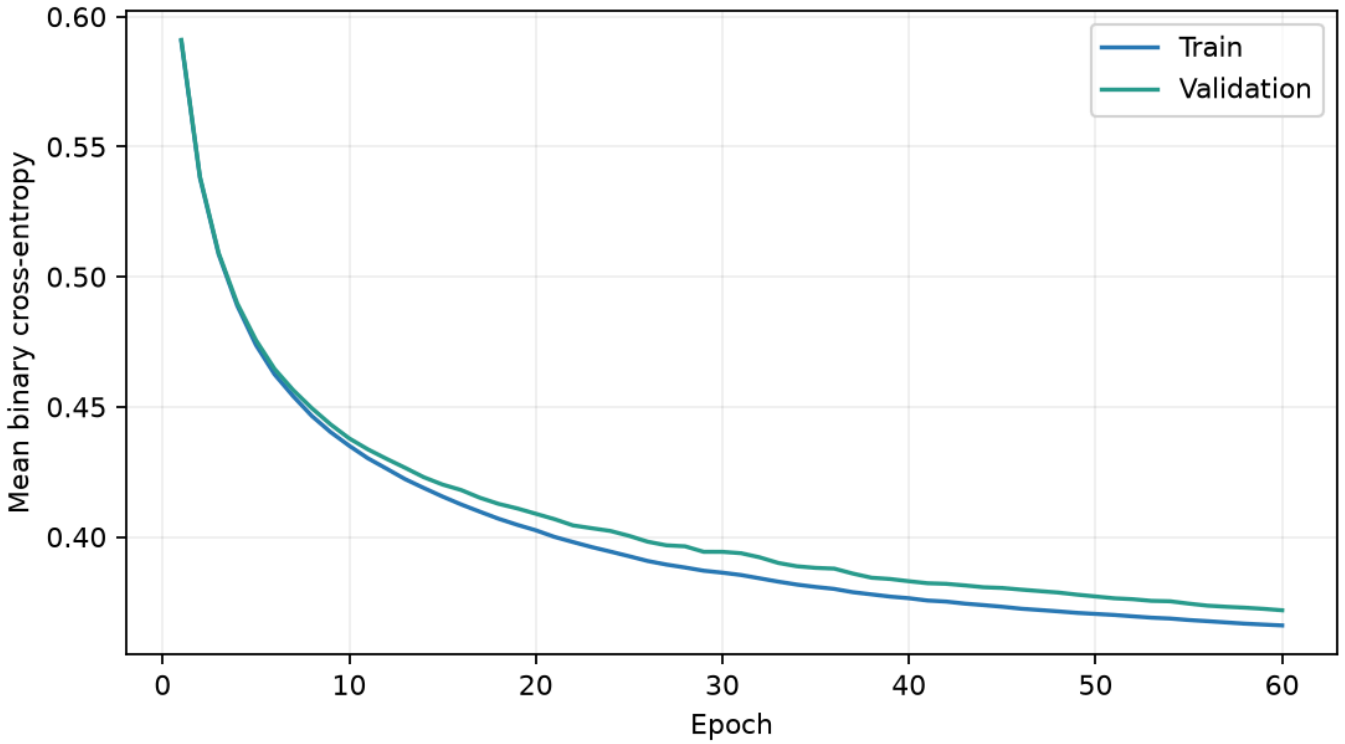


图 1 中训练损失由第 1 轮的 0.5906 降到第 60 轮的 0.3660，验证损失由 0.5911 降到 0.3718。两条曲线均表示不含正则项的数据损失；对于带正则的方案，实际训练总目标还会加上 $\lambda\Omega(\theta)$，因此损失曲线不能直接等同于训练总目标。

5.3 测试集结果

表 6 冻结模型的测试集指标

指标	结果
准确率	0.8411

指标	结果
精确率	0.8118
召回率	0.6832
F1 分数	0.7419
训练多数类基线的测试准确率	0.6656

表 7 测试集混淆矩阵

	预测为 0	预测为 1
真实为 0	TN = 185	FP = 16
真实为 1	FN = 32	TP = 69

测试集共 302 条，其中正类 101 条、负类 201 条。模型正确识别 69 条正类，漏判 32 条，同时把 16 条负类误报为正类。准确率 0.8411 高于训练多数类基线的 0.6656，说明模型不只依赖多数类；但召回率 0.6832 表明仍有约三分之一的真实前 500 名样本未被识别。

5.4 选做：固定 L2 正则下的特征对照

未完成选做，不影响必做部分。主实验冻结模型、完整特征集与阈值均未因选做而改变。

六、结果分析与讨论

6.1 正则化的作用

三种方案的验证准确率均为 0.83，但验证数据损失从无正则的 0.371792 上升到 L2 的 0.379552 和 L1 的 0.392744。正则化在本实验中未改善验证表现。其原因可能是样本量相对于 29 维模型参数并不算很小，且训练轮数固定为 60，主要误差更可能来自特征与标签之间的不可完全分离关系，而不是严重的参数过拟合。L1 和 L2 都引入了额外约束，使模型在训练集上的拟合受到限制。该结果说明不能预设正则化一定提高分类性能，正则化系数和训练设置需要通过验证集判断。

6.2 与多数类基线的比较

多数类基线把全部测试样本预测为负类，准确率为 0.6656。冻结模型的准确率为 0.8411，提高约 17.55 个百分点；精确率为 0.8118，说明被模型判为正类的样本中约 81.18% 确实是前 500 名；召回率为 0.6832，说明 101 条真实正类中识别出 69 条。16 个 FP 表示本不属于前 500 名却被告警的学校，32 个 FN 表示真正属于前 500 名却未被识别的学校。若实际应用更看重不漏掉目标学校，则召回率偏低是主要问题；若误报成本更高，则当前 0.5 阈值下的较高精确率更有价值。

6.3 阈值的取舍

验证集上三个阈值的结果如下：

阈值	TN	FP	FN	TP	准确率	精确率	召回率	F1
0.3	163	37	18	82	0.8167	0.6891	0.8200	0.7489
0.5	180	20	31	69	0.8300	0.7753	0.6900	0.7302
0.7	197	3	44	56	0.8433	0.9492	0.5600	0.7044

阈值从 0.5 降到 0.3 后，召回率由 0.6900 升到 0.8200，但 FP 从 20 增加到 37，精确率由 0.7753 降到 0.6891。阈值升到 0.7 后，FP 只有 3，精确率达到 0.9492，但召回率降到 0.5600，漏判增至 44。可见降低阈值更偏向找出正类，提高阈值更偏向减少误报。主实验预先固定阈值 0.5，不能因为查看测试结果后再调整阈值；0.3 阈值下验证 F1 略高，只能作为后续按任务代价重新设计评价方案的依据。

6.4 任务局限

本年度的部分单项分数与排名构造存在关联，模型可能利用这些统计关系，而不是学习可迁移的学校质量规律；因此结果不能解释为某个字段对排名有因果关系。数据来自第三方镜像，未逐校核验，类别缺失机制也不一定完全随机。实验采用单次固定划分，验证集只有 300 条，指标会受划分波动影响。类别 `Unknown` 将真实缺失和未见类别合并，虽然保证了列对齐，但丢失了两者之间的区别。模型用于未来年份时，还需要重新处理数据时点、类别变化和分布漂移，不能直接把当前测试指标视为跨年泛化能力。

6.5 特征选择

本次未完成固定 L2 系数下的完整组与精简组对照，因此不对删除 `focus`、`research` 的效果作结论。为避免泄漏而排除名称、行号、排名和总分属于建模前的必要字段筛选，与基于验证表现的可选特征选择不同。

七、实验总结

本实验完成了从表格清洗、训练集统计量拟合、类别编码、张量构造到逻辑回归训练和评价的完整流程。C0–C7 的检查点均通过；手算的一次 SGD 更新与程序输出一致，验证了梯度清除、反向传播和参数更新的顺序。三种正则化方案中，无正则方案取得最低验证损失，最终冻结模型在测试集上取得 0.8411 的准确率、0.8118 的精确率、0.6832 的召回率和 0.7419 的 F1，优于 0.6656 的多数类基线。

实验中遇到的主要困难是训练循环的控制流和 notebook 的变量依赖：若把返回语句放在批次循环内部，函数只会处理第一批并返回 `None`；若在未执行 C2 单元时运行 C3 演示，`train` 尚未定义。通过按阶段运行、检查变量来源和保持“预处理—数据集—模型—训练—评价”的顺序，问题得到修正。后续可增加交叉验证和多种随机划分，单独分析缺失机制与 Unknown 类别，并在明确误报和漏判代价后设计成本敏感的阈值选择方案。